

Sorting Algorithms Pseudocode

A clean transcription of the six sorting algorithms shown in the seven handwritten photos. Array indices are zero-based throughout.

The photos do not include pseudocode for the Problem 7 comparison program.

Correctness clarifications

The insertion-sort condition is $j \geq 0$ so index 0 is checked. Sift-down stops when the parent is $\geq$ its larger child. These clarify boundary conditions without changing the intended algorithms.

1. Bubble Sort

Bubble Sort is stable. After each pass, the largest value in the unsorted section reaches its final position. Stop early if a pass makes no swaps.

Pseudocode

```
ALGORITHM BUBBLE-SORT(A)
  n ← length(A)
  for pass ← 0 to n - 2 do
    swapped ← false
    for i ← 0 to n - pass - 2 do
      if A[i] > A[i + 1] then
        swap(A[i], A[i + 1])
        swapped ← true
      end if
    end for
    if swapped = false then
      break
    end if
  end for
  return A
END ALGORITHM
```

Notes

Worst-case time: $O(n^2)$. Stable; equal elements are not swapped.

Bubble Sort - Experimental Results

Execution output for the original array and an already sorted input.

```
===== Problem 1: Bubble Sort =====  
Original: 34 7 23 32 5 62 14 19  
Pass 1: 7 23 32 5 34 14 19 62  
Pass 2: 7 23 5 32 14 19 34 62  
Pass 3: 7 5 23 14 19 32 34 62  
Pass 4: 5 7 14 19 23 32 34 62  
Pass 5: 5 7 14 19 23 32 34 62  
Sorted: 5 7 14 19 23 32 34 62  
Comparisons: 25 Swaps: 15  
Pass 1: 5 7 14 19 23 32 34 62  
Already sorted input: comparisons = 7, swaps = 0  
Process exited with status 0
```

Original input: 25 comparisons and 15 swaps; sorted input: 7 comparisons and 0 swaps.

2. Insertion Sort

Keep the left portion sorted. Take the next value as key, shift larger values one position right, then insert key into the open position.

Pseudocode

```
ALGORITHM INSERTION-SORT(A)
  n ← length(A)
  for i ← 1 to n - 1 do
    key ← A[i]
    j ← i - 1
    while j ≥ 0 and A[j] > key do
      A[j + 1] ← A[j]
      j ← j - 1
    end while
    A[j + 1] ← key
  end for
  return A
END ALGORITHM
```

Notes

Worst-case time: $O(n^2)$. Stable; only values greater than key are shifted.

Insertion Sort - Experimental Results

Execution output from the provided program run.

```
===== Problem 2: Insertion Sort =====  
Original: 34 7 23 32 5 62 14 19  
i = 1, key = 7: 7 34 23 32 5 62 14 19  
i = 2, key = 23: 7 23 34 32 5 62 14 19  
i = 3, key = 32: 7 23 32 34 5 62 14 19  
i = 4, key = 5: 5 7 23 32 34 62 14 19  
i = 5, key = 62: 5 7 23 32 34 62 14 19  
i = 6, key = 14: 5 7 14 23 32 34 62 19  
i = 7, key = 19: 5 7 14 19 23 32 34 62  
Sorted: 5 7 14 19 23 32 34 62  
Comparisons: 20 Shifts: 15  
Sorted input shifts (fewest): 0  
Reverse-sorted input shifts (most): 28  
Process exited with status 0
```

Original input: 20 comparisons and 15 shifts; sorted input: 0 shifts; reverse-sorted input: 28 shifts.

3. Selection Sort

On each pass, find the minimum value in the unsorted portion and swap it with the first value in that portion.

Pseudocode

```
ALGORITHM SELECTION-SORT(A)
  n ← length(A)
  for i ← 0 to n - 2 do
    minIdx ← i
    for j ← i + 1 to n - 1 do
      if A[j] < A[minIdx] then
        minIdx ← j
      end if
    end for
    if minIdx ≠ i then
      swap(A[i], A[minIdx])
    end if
  end for
  return A
END ALGORITHM
```

Notes

Worst-case time: $O(n^2)$. Avoids self-swaps; standard Selection Sort is not stable.

Selection Sort - Experimental Results

Execution output from the provided program run.

```
===== Problem 3: Selection Sort =====  
Original: 34 7 23 32 5 62 14 19  
Pass 1: min = 5 at index 4 -> 5 7 23 32 34 62 14 19  
Pass 2: min = 7 at index 1 -> 5 7 23 32 34 62 14 19  
Pass 3: min = 14 at index 6 -> 5 7 14 32 34 62 23 19  
Pass 4: min = 19 at index 7 -> 5 7 14 19 34 62 23 32  
Pass 5: min = 23 at index 6 -> 5 7 14 19 23 62 34 32  
Pass 6: min = 32 at index 7 -> 5 7 14 19 23 32 34 62  
Pass 7: min = 34 at index 6 -> 5 7 14 19 23 32 34 62  
Sorted:   5 7 14 19 23 32 34 62  
Comparisons: 28 Swaps: 5  
Selection Sort performs at most one swap per pass, so it normally uses fewer swaps than Bubble Sort on this input.  
Process exited with status 0
```

Original input: 28 comparisons and 5 swaps.

4. Quick Sort (Lomuto Partition)

Use the last element of each subarray as pivot. Partition moves values no greater than the pivot to its left and returns the pivot's final index.

Pseudocode

```
ALGORITHM QUICK-SORT(A, low, high)
  if low < high then
    p <- PARTITION(A, low, high)
    QUICK-SORT(A, low, p - 1)
    QUICK-SORT(A, p + 1, high)
  end if
END ALGORITHM
```

```
ALGORITHM PARTITION(A, low, high)
  pivot <- A[high]
  i <- low - 1
  for j <- low to high - 1 do
    if A[j] <= pivot then
      i <- i + 1
      swap(A[i], A[j])
    end if
  end for
  swap(A[i + 1], A[high])
  return i + 1
END ALGORITHM
```

Notes

Worst-case time: $O(n^2)$. A last-element pivot can cause this on sorted input.
Quick Sort is usually unstable.

Quick Sort - Experimental Results

Lomuto partition trace, sorted-input comparison count, and pivot discussion.

```
Choose a problem (1-7): 4

===== Problem 4: Quick Sort (Lomuto) =====
Original: 34 7 23 32 5 62 14 19
pivot = 19, subarray: 34 7 23 32 5 62 14 19
swap i=0, j=1: 7 34 23 32 5 62 14 19
swap i=1, j=4: 7 5 23 32 34 62 14 19
swap i=2, j=6: 7 5 14 32 34 62 23 19
pivot 19 final index = 3: 7 5 14 19 34 62 23 32
  pivot = 14, subarray: 7 5 14
    pivot = 5, subarray: 7 5
      pivot 5 final index = 0: 5 7
    pivot = 32, subarray: 34 62 23 32
      swap i=4, j=6: 23 62 34 32
      pivot 32 final index = 5: 23 32 34 62
        pivot = 62, subarray: 34 62
Sorted:  5 7 14 19 23 32 34 62
Comparisons: 14 Swaps: 7
Already sorted input comparisons: 28
Last-element pivots produce unbalanced partitions on sorted data; randomized or median-of-three pivots reduce this risk.
Process exited with status 0
```

Original input: 14 comparisons and 7 swaps; sorted input: 28 comparisons.

5. Merge Sort

Recursively split the array, then merge the two sorted halves. When values are equal, take the left-half value first ($\leq$), preserving stability.

Pseudocode

```
ALGORITHM MERGE-SORT(A, low, high)
  if low < high then
    mid <- floor((low + high) / 2)
    MERGE-SORT(A, low, mid)
    MERGE-SORT(A, mid + 1, high)
    MERGE(A, low, mid, high)
  end if
END ALGORITHM

ALGORITHM MERGE(A, low, mid, high)
  L <- copy of A[low .. mid]
  R <- copy of A[mid + 1 .. high]
  i <- 0; j <- 0; k <- low
  while i < length(L) and j < length(R) do
    if L[i] <= R[j] then
      A[k] <- L[i]
      i <- i + 1
    else
      A[k] <- R[j]
      j <- j + 1
    end if
    k <- k + 1
  end while
  while i < length(L) do
    A[k] <- L[i]
    i <- i + 1; k <- k + 1
  end while
  while j < length(R) do
    A[k] <- R[j]
    j <- j + 1; k <- k + 1
  end while
END ALGORITHM
```

Notes

Worst-case time: $O(n \log n)$. Stable; equal values are taken from the left half first.

Merge Sort - Experimental Results

Execution output from the provided program run.

```
Choose a problem (1-7): 5

===== Problem 5: Merge Sort =====
Original: 34 7 23 32 5 62 14 19
split: 34 7 23 32 5 62 14 19
split: 34 7 23 32
split: 34 7
merged: 7 34
split: 23 32
merged: 23 32
merged: 7 23 32 34
split: 5 62 14 19
split: 5 62
merged: 5 62
split: 14 19
merged: 14 19
merged: 5 14 19 62
merged: 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Comparisons while merging: 17
Sorted-input comparisons: 12
Reversed-input comparisons: 12
Extra memory: O(n) merge buffer plus O(log n) recursion stack.
Process exited with status 0
```

Merge comparisons: 17 (sorted input: 12; reversed input: 12).

6. Heap Sort

Build a max-heap, then move the maximum at the root to the end and sift down the root of the smaller heap.

Pseudocode

```
ALGORITHM HEAP-SORT(A)
  n <- length(A)
  for i <- floor(n / 2) - 1 downto 0 do
    SIFT-DOWN(A, i, n)
  end for
  for end <- n - 1 downto 1 do
    swap(A[0], A[end])
    SIFT-DOWN(A, 0, end)
  end for
  return A
END ALGORITHM

ALGORITHM SIFT-DOWN(A, root, size)
  while 2 * root + 1 < size do
    child <- 2 * root + 1
    if child + 1 < size and A[child + 1] > A[child] then
      child <- child + 1
    end if
    if A[root] >= A[child] then
      break
    end if
    swap(A[root], A[child])
    root <- child
  end while
END ALGORITHM
```

Notes

Worst-case time: $O(n \log n)$. In-place; standard Heap Sort is not stable.

Heap Sort - Experimental Results

Execution output from the provided program run.

```
===== Problem 6: Heap Sort =====
Original: 34 7 23 32 5 62 14 19
Heap: 62 32 34 19 5 23 14 7
end = 7: heap: 34 32 23 19 5 7 14
sorted: 62
end = 6: heap: 32 19 23 14 5 7
sorted: 34 62
end = 5: heap: 23 19 7 14 5
sorted: 32 34 62
end = 4: heap: 19 14 7 5
sorted: 23 32 34 62
end = 3: heap: 14 5 7
sorted: 19 23 32 34 62
end = 2: heap: 7 5
sorted: 14 19 23 32 34 62
end = 1: heap: 5
sorted: 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Swaps in siftDown: 12
Heap Sort is not stable: swaps can change the relative order of equal values.
Process exited with status 0
```

Sift-down swaps: 12. The output notes that Heap Sort is not stable.

Problem 7 - Comparing the Six Algorithms

Pseudocode

Use the six implementations from Problems 1-6 without their step-by-step trace output. Count comparisons on the required input patterns, then benchmark each algorithm on identical random data for each size.

```
ALGORITHM COMPARE-SIX-SORTS
  A <- [34, 7, 23, 32, 5, 62, 14, 19]
  SORTS <- [Bubble, Insertion, Selection, Quick, Merge, Heap]
  INPUTS <- [A, sorted copy of A, reverse-sorted copy of A]

  for each input in INPUTS do
    for each sort in SORTS do
      values <- copy(input)
      comparisons <- 0
      run sort(values) with step-by-step tracing disabled
      print sort name, input name, comparisons
      print first min(10, length(values)) values using printArray
    end for
  end for

  for n in [1000, 5000, 10000] do
    randomData <- n random integers in [1, 100000]
    for each sort in SORTS do
      values <- copy(randomData)          // same data for every sort
      start <- high-resolution clock
      run sort(values) with tracing disabled
      elapsed <- stop clock - start
      print n, sort name, elapsed time
      print first min(10, n) values using printArray
    end for
  end for

  compare measured counts and times with expected growth rates
  identify algorithms with quadratic and n log n growth
  explain whether measurements agree with theory
  recommend a sort for large random and nearly sorted inputs
END ALGORITHM
```

Analysis to report

State which methods have $O(n^2)$ worst-case growth and which have $O(n \log n)$ growth; Quick Sort is $O(n \log n)$ on average and $O(n^2)$ in the worst case. Compare the measured timings with this theory and explain choices for large random versus nearly sorted input.

Problem 7 - Comparing the Six Algorithms

Execution output from the provided program run.

```
Choose a problem (1-7): 7

===== Problem 7: Comparing the Six Algorithms =====
Comparison counts; each row also prints the first 10 sorted values.

Bubble
Input      Comparisons  First 10 values
Original    25  5 7 14 19 23 32 34 62
Sorted      7   5 7 14 19 23 32 34 62
Reversed    27  5 7 14 19 23 32 34 62

Insertion
Input      Comparisons  First 10 values
Original   20  5 7 14 19 23 32 34 62
Sorted     7   5 7 14 19 23 32 34 62
Reversed   18  5 7 14 19 23 32 34 62

Selection
Input      Comparisons  First 10 values
Original   28  5 7 14 19 23 32 34 62
Sorted    28  5 7 14 19 23 32 34 62
Reversed   28  5 7 14 19 23 32 34 62

Quick
Input      Comparisons  First 10 values
Original   14  5 7 14 19 23 32 34 62
Sorted    28  5 7 14 19 23 32 34 62
Reversed   18  5 7 14 19 23 32 34 62

Merge
Input      Comparisons  First 10 values
Original   17  5 7 14 19 23 32 34 62
Sorted    12  5 7 14 19 23 32 34 62
Reversed   17  5 7 14 19 23 32 34 62

Heap
Input      Comparisons  First 10 values
Original   12  5 7 14 19 23 32 34 62
Sorted    15  5 7 14 19 23 32 34 62
Reversed   12  5 7 14 19 23 32 34 62

Timing random arrays (milliseconds)
n      Bubble   Insertion   Selection   Quick      Merge      Heap
1000   7.262     1.797       3.182       0.140      0.215      0.247
5000   84.629    17.284      29.753      0.354      0.574      0.592
10000  247.866     68.729     118.213     0.858      1.229      1.292

Theory: Bubble, Insertion, Selection are  $O(n^2)$ ; Merge and Heap are  $O(n \log n)$ ; Quick is  $O(n \log n)$  average and  $O(n^2)$  worst case.
Merge or Heap is predictable for large random arrays; Insertion can be effective for nearly sorted data.
Process exited with status 0
```

Comparison counts, sorted-output checks, and timing results for $n = 1,000, 5,000, \text{ and } 10,000$.